Shopping List
Architecture and Verification

Technical overview | 15 October 2026
Creator: Dmitry Fridman

A Hebrew, right-to-left shopping application with two React pages and two independent backend services. SQL Server holds the product catalog; MongoDB holds submitted orders.

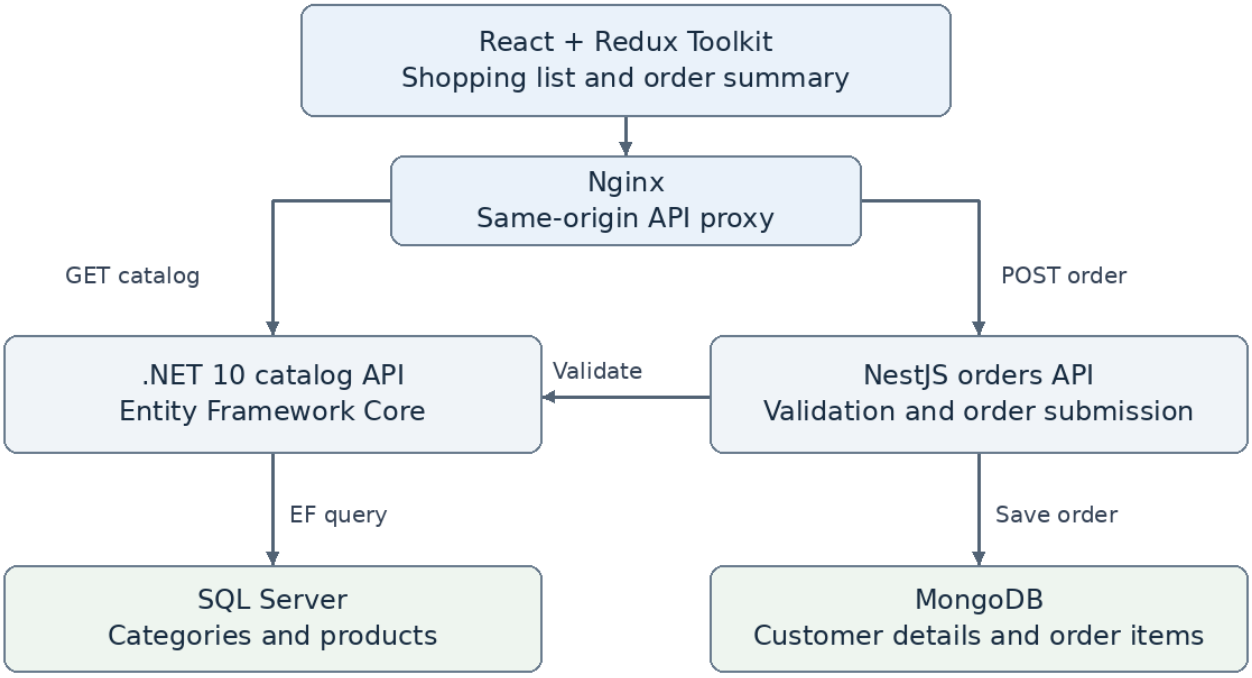


Figure 1. Application flow. The accompanying .drawio file is editable in diagrams.net.

How the flow works

1. Opening the shopping page loads categories and products from the .NET API. Entity Framework Core reads SQL Server.

2. The user selects products and quantities. Redux Toolkit holds the cart and passes it to the order summary page.

3. The user enters a Hebrew full name and address, plus an email address. The frontend sends the details and cart to NestJS.

4. NestJS validates the request, checks the products against the catalog API and saves the order in MongoDB. A successful response returns the order ID.

Implementation and automated checks

Part	Source location	Responsibility
Frontend	apps/web	React, Redux Toolkit and Hebrew RTL forms
Catalog service	apps/catalog-api	ASP.NET Core 10, EF Core and SQL Server
Order service	apps/orders-api	NestJS, Mongoose and MongoDB
Infrastructure	compose.yaml; scripts/	Container startup, database setup and checks

Nginx serves the frontend and proxies API calls. A one-time catalog initialization task creates and seeds the SQL database with 4 categories and 12 products. Its successful exit is expected. Database volumes retain data between container restarts.

Recorded automated results

Implementation verification recorded on 12 October 2026 in docs/VERIFICATION.md:

Check	Recorded result	Coverage
Frontend tests	12 passed	Cart, Hebrew validation, HTTP failures and page behavior
NestJS tests	14 passed	DTO validation, order handling, retry behavior and safe HTTP errors
.NET tests	4 passed	EF model, seeded catalog, relationships and HTTP errors
Production builds	Passed	React/Vite, NestJS and .NET Release publish

Total: 30 automated tests passed. npm and NuGet dependency checks reported no known vulnerabilities at verification time.

Test scope: .NET relational tests use SQLite in memory. NestJS tests substitute database and catalog dependencies. These automated tests do not by themselves prove live SQL Server or MongoDB persistence. The next page contains the separate local runtime evidence.

Reproduce the checks

From the extracted repository root, with the documented prerequisites installed and the Docker stack running:

```
npm ci
npm test
npm run test:dotnet
node scripts/verify.mjs
```

The verify script writes one sample order. To inspect its stored document, run node scripts/inspect-databases.mjs ORDER_ID with the returned ID. See docs/TESTING.md for database and API checks.

Evidence of a successful local run

The following terminal captures were supplied during the local run on 12 October 2026. They document that run; they are not a new execution on the report date.

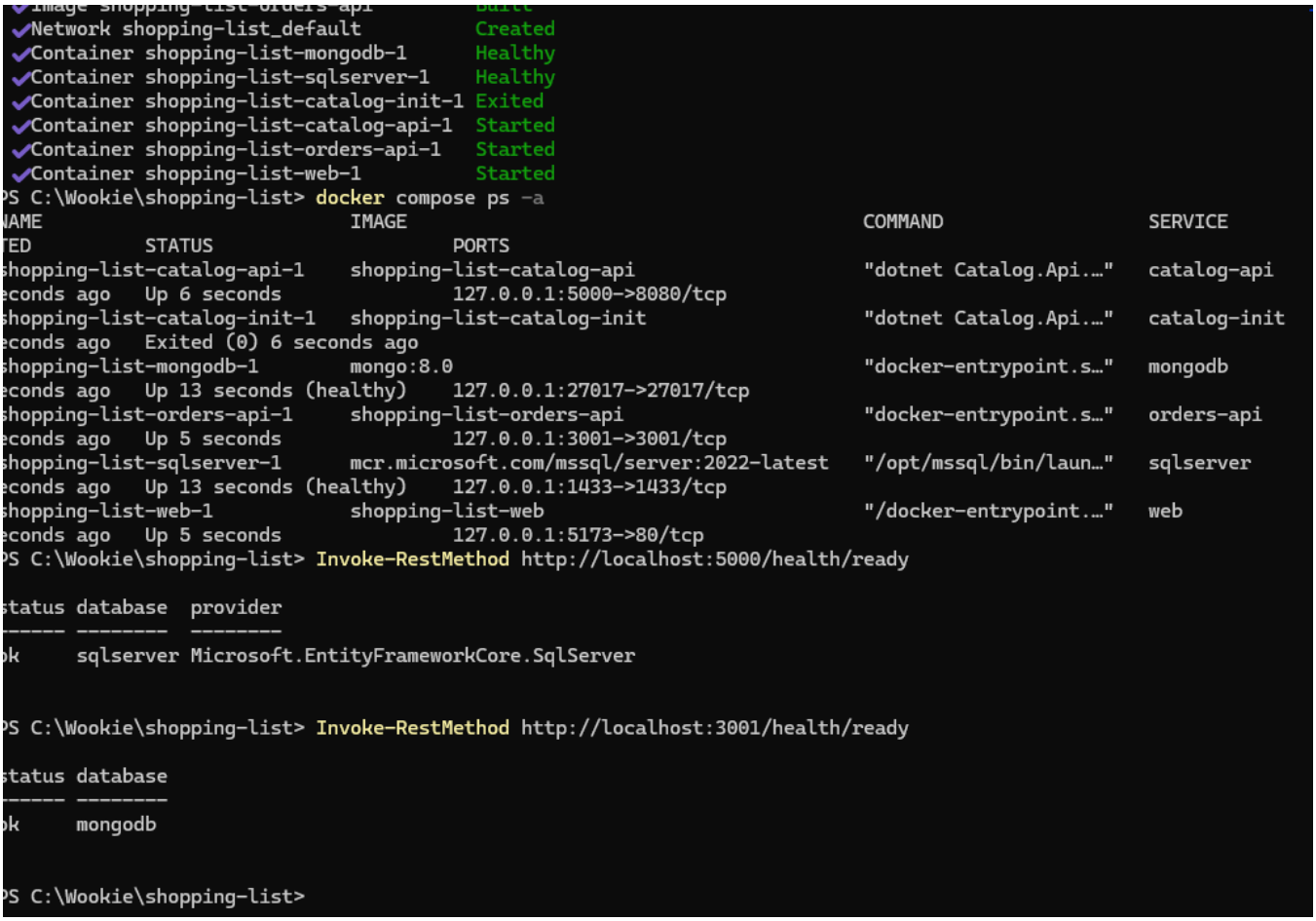


Figure 2. Docker services running, catalog initialization exited with code 0, and readiness endpoints returned SQL Server / EF Core and MongoDB status.

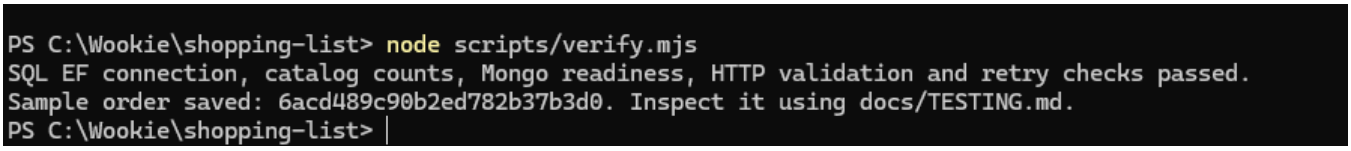


Figure 3. The live verification script completed successfully and returned a sample order ID.

The successful script checks both database readiness endpoints, the SQL Server EF provider, 4 categories and 12 products, rejection of an English full name (HTTP 400), order submission (HTTP 201), and a repeated request returning the same order ID.

Sample order ID: 6acd489c90b2ed782b37b3d0

Evidence scope: the captures demonstrate service startup and API-level checks. Direct inspection of the saved MongoDB document and a complete visual browser walkthrough are separate checks.